

From Markov to Laplace: How Mamba In-context Learns Markov chains

Anonymous Authors¹

Abstract

While transformer-based language models have driven the AI revolution thus far, their computational complexity has spurred growing interest in viable alternatives, such as structured state space sequence models (SSMs) and Selective SSMs. Among these, Mamba (S6) and its variant Mamba-2 have shown remarkable inference speed ups over transformers while achieving comparable or superior performance on complex language modeling tasks. However, despite these architectural innovations and empirical successes, the fundamental learning capabilities of Mamba remain poorly understood. In this paper, we address this gap by studying in-context learning (ICL) on Markov chains and uncovering a surprising phenomenon: unlike transformers, even a single-layer Mamba efficiently learns the in-context Laplacian smoothing estimator, which is both Bayes and minimax optimal, for all Markovian orders. To explain this, we theoretically characterize the representation capacity of Mamba and reveal the fundamental role of convolution in enabling it to represent the optimal Laplacian smoothing. These theoretical insights align strongly with empirical results and, to the best of our knowledge, represent the first formal connection between Mamba and optimal statistical estimators. Finally, we outline promising research directions inspired by these findings. Code is available at <https://anonymous.4open.science/r/Markov-Mamba-39C5>.

1. Introduction

Transformers have been at the forefront of recent breakthroughs in language modeling, driving the AI revolution (Vaswani et al., 2017; Radford & Narasimhan, 2018; Devlin

et al., 2018). Despite their empirical success, transformers suffer from high computational complexity, such as quadratic scaling in sequence length during training and linear cache size at inference (Gu & Dao, 2023a). To address these limitations, there is a growing interest in designing alternative efficient architectures among which structured state space models (SSMs) are the most prominent. In particular, Selective SSMs such as Mamba and Mamba-2, have achieved state-of-the-art results in various language modeling tasks, while greatly improving the inference throughput (Cirone et al., 2025).

Motivated by this success, there is tremendous interest in understanding the sequential modeling abilities of SSMs, especially that of Mamba. In particular, a growing body of research explores Mamba through its in-context learning (ICL) capabilities (Grazzi et al., 2024; Halloran et al., 2024; Akyürek et al., 2024), mirroring a theme that has been successful in unraveling fundamental mechanisms (e.g. induction heads) behind transformers (Nichani et al., 2024; Rajaraman et al., 2024). While these works reveal interesting insights about Mamba’s ICL abilities vis-a-vis transformers, they are largely empirical in nature, and we currently lack a fundamental understanding of Mamba and its underlying learning mechanisms. We are thus motivated to ask: *Can we systematically characterize the ICL capabilities of Mamba?*

To address this, in this paper we introduce a principled framework for theoretical and empirical analysis of Mamba’s ICL capabilities via random Markov chains. Leveraging this framework, we uncover a surprising phenomenon: unlike transformers, even a single-layer Mamba efficiently learns the in-context Laplacian smoothing estimator, which is both Bayes and minimax optimal, for all Markovian orders (Fig. 1). Towards explaining this, we theoretically characterize the representation capacity of Mamba and demonstrate that the convolution mechanism plays a fundamental role in realizing the Laplacian smoothing. Further we showcase that these theoretical insights align strongly with empirical results, both on Markov and complex-natural language data. To the best of our knowledge, this is the first-result-of-its-kind connecting Mamba and optimal statistical estimators.

In summary, we make the following contributions:

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

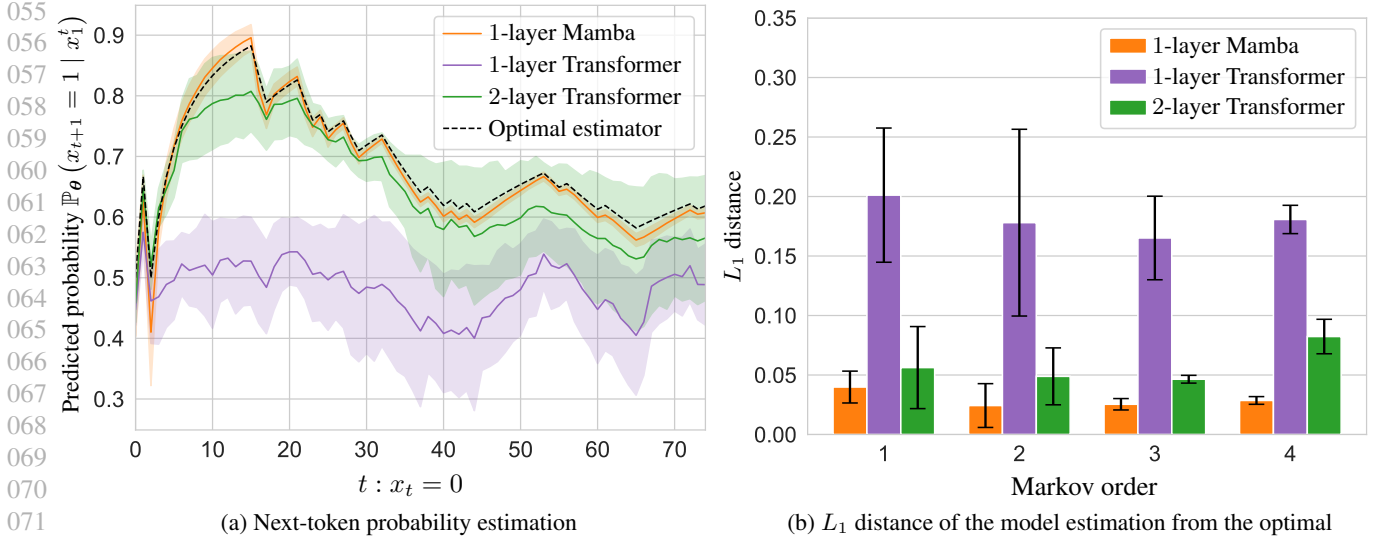


Figure 1. Single-layer Mamba learns the optimal Laplacian smoothing when trained on random Markov chains, exhibiting in-context learning. A two-layer transformer also learns the same, albeit less precisely. In contrast, a single-layer transformer fails to solve this task. We observe the same phenomenon for various Markov orders.

- We provide a novel framework for a precise theoretical and empirical study of Mamba’s ICL via random Markov chains (Sec. 2.1).
- Capitalizing our framework, we uncover the surprising fact that convolution mechanism plays a pivotal role in Mamba’s learning abilities (Sec. 3).
- Building upon these insights, we characterize the representational capacity of single-layer Mamba and demonstrate that it can represent the optimal Laplacian smoothing estimator for first-order processes (Sec. 4).
- We demonstrate the generality of our findings on non-Markovian data and illustrate the fundamental role of convolution even on complex language-modeling tasks (Sec. 5).

Notation. Scalars are denoted by such italic lower case letters as x, y , Euclidean vectors by bold $\mathbf{x}, \mathbf{y}$, and matrices by upper case X, Y , etc. $\|\cdot\|_p$ denotes the ℓ_p -norm for $p \in [1, \infty]$. $\mathbf{1}$ refers to the all-one vector. For $T \in \mathbb{N}$, $[T] \triangleq \{1, \dots, T\}$, and for a sequence $(x_t)_{t \geq 1}$, define $x_k^t \triangleq (x_k, \dots, x_t)$ if $k \geq 1$ and $(x_1, \dots, x_t)$ otherwise. For $z \in \mathbb{R}$, $\text{sigmoid}(z) \triangleq 1/(1+e^{-z})$, $\text{ReLU}(z) \triangleq \max(0, z)$, and $\text{softplus}(z) \triangleq \log(1 + e^z)$. $\text{Unif}(S)$ denotes the uniform distribution over a set S and $\text{Dir}(\beta)$ denotes the Dirichlet distribution with parameter $\beta > 0$. $D_{\text{KL}}(P\|Q)$ denotes the KL-divergence between distributions P and Q .

2. Problem setup

2.1. Input data: Random Markov chains

To investigate the ICL capabilities of Mamba, we let the input tokens to be stochastic and drawn from a random Markov chain of order k (Edelman et al., 2024). That is, the token sequence $x = (x_t)_{t=1}^T \in \mathcal{X}^T$ on the state space (vocabulary) $\mathcal{X}$ follows the transition dynamics:

$$\mathbb{P}(x_{t+1} = \cdot | x_1^t) = \mathbb{P}(x_{t+1} = \cdot | x_{t-k+1}^t), \quad (1)$$

almost surely for all $t \in [T]$, and the k^{th} -order Markov kernels, $\mathbb{P}(x_{t+1} = \cdot | x_{t-k+1}^t = i_{t-k+1}^t)$, are sampled independently for each tuple $(i_{t-k+1}, \dots, i_t)$ from the Dirichlet prior $\text{Dir}(\beta \cdot \mathbf{1})$, with $\beta > 0$. When $\beta = 1$, this corresponds to the uniform distribution on the S -dimensional simplex Δ_1^S , where size $S = |\mathcal{X}|$.

The transition matrix $P = (P_{i_1^k, \cdot}) \in (0, 1)^{S^k \times S}$ encapsulates the set of all S^k possible kernels with each row corresponding to the aforementioned conditional distribution. While this transition matrix governs the generation of each token x_t for $t > k$, the first k -tokens $x_1, \dots, x_k$ are drawn i.i.d. from $\text{Unif}(\mathcal{X})$. This constitutes the joint law of the random variables (P, x) , termed random Markov distribution henceforth. More succinctly,

Data generation (Random Markov sequences).

1. Draw P with each row sampled i.i.d. from $\text{Dir}(\beta \cdot \mathbf{1})$.
2. For $t = 1, \dots, k$, sample $x_t \sim \text{Unif}(\mathcal{X})$.
3. For $t = k, \dots, T$, sample $x_{t+1} \sim P_{x_{t-k+1}^t, \cdot}$.

4. Return the input $x = (x_t)_{t=1}^T$.
5. Repeat the above steps to generate a batch $\{x^{(b)}\}_{b \in [B]}$.

Relation to ICL. As a consequence of the generation process, no two sequences x share the same transition matrix P and hence every sequence follows a different Markov distribution. Owing to this fact, even when a model is trained on this random Markovian data, at inference, for every test sequence it has to estimate the next token in-context. Hence this data class serves as a good sandbox to gauge the ICL capabilities of Mamba, which was also used in a similar context for transformers (Nichani et al., 2024). In this paper, we let the state space $\mathcal{X} = \{0, 1\}$ for the ease of exposition and the order k to be any $k \geq 1$. Our results and insights can be readily extended to any finite vocabulary, as demonstrated for the natural language in Sec. 5.2.

2.2. Mamba architecture

Selective SSMs such as Mamba and Mamba-2 are a class of sequence-to-sequence models that are closely related to RNNs and classical state space models (Gu & Dao, 2023b). A key feature underpinning these models is the *selectivity mechanism*, enabling them to selectively choose inputs at every timestep, as opposed to linear time-invariant (LTI) systems. While we believe our work captures the behavior of all selective SSMs, we will specifically focus on the state-of-the-art Mamba-2 model to simplify exposition. By slight abuse of terminology, henceforth we will also refer to this model simply as Mamba. Mathematically speaking, Mamba implements the sequence-to-sequence mapping $\text{Mamba} : \mathbb{R}^{d \times T} \mapsto \mathbb{R}^{d \times T}$, where given a sequence of input embeddings $x = (x_t)_{t=1}^T \in \mathbb{R}^{d \times T}$ of dimension d , it outputs the corresponding output embeddings $o = (o_t)_{t=1}^T \in \mathbb{R}^{d \times T}$ of the same dimension with $o = \text{Mamba}(x)$. More precisely, fix $t \in [T]$. Then the output o_t at time t is computed as $o_t = \text{Mamba}(x_1^t)$ using the following recurrence equations (Dao & Gu, 2024):

$$\begin{aligned} H_t &= a_t H_{t-1} + \tilde{x}_t b_t^\top \in \mathbb{R}^{ed \times N}, \\ y_t &= H_t c_t \in \mathbb{R}^{ed}, \\ z_t &= y_t \odot \text{ReLU}(W_z x_t) \in \mathbb{R}^{ed}, \\ o_t &= W_o z_t \in \mathbb{R}^d, \end{aligned} \quad (\text{Mamba})$$

where the initial state $H_0 = 0$, $W_z \in \mathbb{R}^{ed \times d}$ and $W_o \in \mathbb{R}^{d \times ed}$ are learnable parameters, and the input-selective

$$\begin{aligned} a_t &\triangleq \exp(-a \cdot \Delta_t) \in (0, 1), \text{ with} \\ \Delta_t &\triangleq \text{softplus}(\langle w_\Delta, x_t \rangle + \delta) \in \mathbb{R}, \\ \tilde{x}_t &\triangleq \text{ReLU}(\text{conv}_X(W_X x_{t-w+1}^t)) \cdot \Delta_t \in \mathbb{R}^{ed}, \\ b_t &\triangleq \text{ReLU}(\text{conv}_B(W_B x_{t-w+1}^t)) \in \mathbb{R}^N, \\ c_t &\triangleq \text{ReLU}(\text{conv}_C(W_C x_{t-w+1}^t)) \in \mathbb{R}^N, \end{aligned} \quad (2)$$

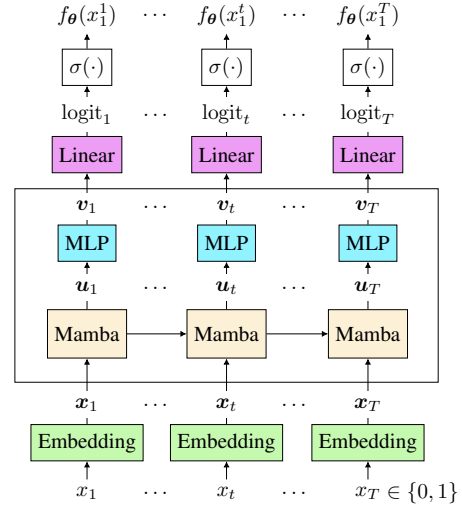


Figure 2. Mamba-based language model with binary input data: for each $t \in [T]$, the next-token prediction probability is $f_\theta(x_1^t) = \mathbb{P}_\theta(x_{t+1} = \cdot \mid x_1^t)$.

where $a \geq 0$, $w_\Delta \in \mathbb{R}^d$, $\delta \in \mathbb{R}$, $W_X \in \mathbb{R}^{ed \times d}$, $W_B \in \mathbb{R}^{N \times d}$ and $W_C \in \mathbb{R}^{N \times d}$ are all learnable parameters and $\text{conv}(z_{t-w+1}^t)$ is a (learnable) time-wise convolution of window $w \in \mathbb{N}$ with distinct kernels per dimension. Here $e \in \mathbb{N}$ denotes the expansion factor for the features, typically 2. Let θ_{Mamba} denote the set of all these parameters.

Intuition behind Mamba. While the update equations in Mamba might seem complicated at a first glance, the underlying intuition is simple: given a sequence of input embeddings (x_t) , we first capture their local temporal information using three separate convolutions to compute $\tilde{x}_t$, b_t , and c_t respectively (Eq. (2)). Equipped with this local memory, we perform the classical linear state update to compute the current state H_t from the past H_{t-1} , weighed by an input-dependent decay factor $a_t \in (0, 1)$, and $(\tilde{x}_t, b_t)$. Subsequently, we compute the state projection y_t , modulate it with an input-selective ReLU term to yield z_t , and finally project it down to get the d -dimensional output embedding o_t . Thus the output o_t at time t is a function of the entire input sequence till then, x_1^t , yielding $o_t = \text{Mamba}(x_1^t)$.

Mamba-based language model. Mamba block is then incorporated into a full-fledged language model as follows: let $x = (x_1, x_2, \dots, x_T) \in \mathcal{X}^T$ be an input token-sequence over the alphabet $\mathcal{X}$; here $\mathcal{X} = \{0, 1\}$ as explained in Sec. 2.1. Then, at every $t \in [T]$, the output of the language model θ is given by the following sequence of equations (Dao & Gu, 2024):

$$\begin{aligned} x_t &= e_{x_t} = x_t e_1 + (1 - x_t) e_0 \in \mathbb{R}^d, \quad (\text{Embedding}) \\ u_t &= x_t + \text{Mamba}(x_1^t) \in \mathbb{R}^d, \quad (\text{Mamba}) \\ v_t &= u_t + W_2[\text{ReLU}(W_1 u_t)] \end{aligned}$$

$$\odot W_3 \mathbf{u}_t] \in \mathbb{R}^d, \quad (\text{MLP})$$

$$\text{logit}_t = W_\ell \mathbf{v}_t \in \mathbb{R}^2, \quad (\text{Linear})$$

$$\begin{aligned} f_\theta(x_1^t) &\triangleq \mathbb{P}_\theta(x_{t+1} = \cdot \mid x_1^t) \\ &= \text{softmax}(\text{logit}_t) \in [0, 1]^2, \end{aligned} \quad (\text{Prediction})$$

where the parameters $\mathbf{e}_0, \mathbf{e}_1 \in \mathbb{R}^d$, $W_1 \in \mathbb{R}^{4d \times d}$, $W_2 \in \mathbb{R}^{d \times 4d}$ and $W_\ell \in \mathbb{R}^{2 \times d}$ are learnable, and $f_\theta(x_1^t)$ is the probability law for the next symbol x_{t+1} conditioned on the past x_1^t . We omit the layer norm here for simplicity. We compactly denote the set of all model parameters as θ , i.e. $\theta = (\mathbf{e}_0, \mathbf{e}_1, \theta_{\text{Mamba}}, W_{1,2,3}, W_\ell) \in \mathbb{R}^D$.

2.3. Learning task: next-token prediction

With the objective of auto-regressively estimating the next token, we train the model parameters θ to minimize the cross-entropy loss between the next-token predicted probability $f_\theta(x_1^t)$ and the corresponding ground-truth symbol x_{t+1} across all the positions $t \in [T]$:

$$\begin{aligned} L(\theta) &\triangleq -\frac{1}{T} \sum_{t \in [T]} \mathbb{E}_P \mathbb{E}_{x_1^{t+1} \sim P} [x_{t+1} \cdot \log f_\theta^{(1)}(x_1^t) \\ &\quad + (1 - x_{t+1}) \cdot \log f_\theta^{(0)}(x_1^t)], \end{aligned} \quad (3)$$

where the expectation is both over the transition kernels P and the Markov sequences $x = (x_t)_{t=1}^T$ generated from P . In practice, it is replaced by empirical average across a finite set of batches, sampled according to the random Markov distribution in Sec. 2.1. For our experiments we use the AdamW optimizer (Kingma & Ba, 2015).

2.4. Optimal estimator: Laplacian smoothing

Given the Bayesian prediction loss in Eq. (3), it is natural to ask: *what's the optimal θ minimizing it?* It follows from a classical result in statistics ((Rissanen, 1984), § A) that this minimum is achieved when the corresponding model prediction matches the (averaged) ground-truth predictive distribution, i.e. $\mathbb{P}_\theta(x_{t+1} = 1 \mid x_1^t) = \mathbb{E}_{P \mid x_1^t} [\mathbb{P}(x_{t+1} = 1 \mid x_1^t)]$, for all t . Given the joint distribution of the pair (P, x_1^{t+1}) in Sec. 2.1, where the kernel $P \sim \text{Dir}(\beta \cdot \mathbf{1})$, it can be shown (§ A) that the conditional expectation above simplifies to the well-known *Laplacian smoothing*, also known as the *add- β estimator* (see e.g. (Merhav & Feder, 1998)):

$$\begin{aligned} \mathbb{P}_\beta^{(k)}(x_{t+1} = 1 \mid x_1^t) &\triangleq \mathbb{E}_{P \mid x_1^t} [\mathbb{P}(x_{t+1} = 1 \mid x_1^t)] \\ &= \frac{n_1 + \beta}{n + 2\beta}, \end{aligned} \quad (\text{Laplacian-smoothing})$$

where n_1 is the number of times token 1 follows the current k^{th} -order context x_{t-k+1}^t in the sequence x_1^t , i.e. $n_1 = |\{i : (x_{i-k}^{i-1}, x_i) = (x_{t-k+1}^t, 1)\}|$ and n is the frequency of this context, i.e. $n = |\{i : x_{i-k}^{i-1} = x_{t-k+1}^t\}|$. Adjusting these

counts by β plays the role of additive smoothing, which avoids assigning zero probabilities to unseen events, an idea dating back to Laplace (Laplace, 1814). If Mamba realizes this smoothing estimator, i.e. $\mathbb{P}_\theta = \mathbb{P}_\beta^{(k)}$, it automatically implies its ICL abilities: given a fresh test sequence at inference, in order to optimally predict the next token, it processes the input tokens in-context to compute the relevant counts, as in the *Laplacian-smoothing*. *But does it realize in practice this optimal smoothing-based counting estimator?* Indeed, as we demonstrate next.

3. Does Mamba learn in-context estimators?

To investigate the ICL capabilities of Mamba, we consider the problem setup described above and train a single-layer Mamba using AdamW on the next-token prediction loss in Eq. (3) on random Markov chains. We repeat the same procedure for both single and two layer transformers and report the best performance for all these models (we refer to § E for more experimental details). These experiments reveal interesting and rather surprising insights about Mamba:

1. Mamba learns the optimal Laplacian smoothing estimator on the Markov prediction task, even with a single layer (Fig. 1a).
2. Convolution mechanism plays a fundamental role in Mamba, aiding in its learning abilities (Fig. 3a)

In the sequel, we expand upon these observations in detail.

1) Mamba learns the Laplacian smoothing. After training, we evaluate the Mamba and transformer models on the same test sequence fixed beforehand and compare their performance to that of the optimal *Laplacian-smoothing* estimator. Specifically, we compare their next-token prediction probabilities with those of the add- β estimator. Fig. 1 illustrates these results, which uncovers a surprising phenomenon: *even a single-layer Mamba sharply matches the optimal estimator on the whole sequence*. In fact, this approximation error is small even for various Markov orders. On the other hand, for transformers we observe that a two-layer model also matches the predictor, albeit less sharply and more variance, whereas a single layer transformers fails to solve the task. This is not fully surprising, given a recent theoretical result (Sanford et al., 2024) that while an induction head (realizing the counting estimator) can be efficiently represented by a two-layer transformer, its single-layer variant needs to be exponentially large.

2) Convolution is the key. To decipher the key architectural component behind Mamba's success in Markov prediction task, we do an ablation study on its three main features: (i) convolution in Eq. (2), (ii) ReLU non-linearity in Eq. (2), and (iii) the gating mechanism in Mamba and

MLP. Amongst them, surprisingly, *convolution* plays a fundamental role in the model’s performance, as illustrated in Fig. 3a. Here we compare the full Mamba architecture from Sec. 2.2, Mamba with just the convolution in Eq. (2) removed, and a simplified Mamba architecture with only convolution (MambaZero in Sec. 4.1). As a metric of comparison, we use the closeness of each of these models θ to the optimal add- β estimator, i.e. $|L(\theta) - L(\beta)|$, where $L(\beta)$ is the optimal loss incurred by the add- β estimator. Closer this metric is to zero, the better the model’s performance is. Remarkably, the simplified Mamba with just the convolution succeeds on the Markov prediction task, while the full model without convolution fails, highlighting its fundamental importance. This raises a natural question: *How does convolution help Mamba to implement the optimal Laplacian estimator?*

In the next section, we investigate this theoretically.

4. Theoretical results

Motivated by Mamba’s success in learning the optimal smoothing estimator, and convolution’s pivotal role in it, here we study how it can represent Laplacian smoothing.

4.1. MambaZero: Simplified model

Building upon the insight that Mamba with just the convolution achieves the same performance as that of the full model (Fig. 3a), we consider its simplified version: MambaZero. MambaZero retains only the essential elements of the full model in Sec. 2.2: the *Embedding* layer, the convolution inside the Mamba block in Eq. (2), and the *Linear* layer. More formally, it’s given by:

$$\mathbf{x}_t = x_t \mathbf{e}_1 + (1 - x_t) \mathbf{e}_0 \in \mathbb{R}^d, \quad (\text{Embedding})$$

$$\mathbf{u}_t = \mathbf{x}_t + \text{MambaZero}(\mathbf{x}_1^t) \in \mathbb{R}^d, \quad (\text{MambaZero})$$

$$\text{logit}_t = W_\ell \mathbf{u}_t \in \mathbb{R}^2, \quad (\text{Linear})$$

$$\begin{aligned} f_\theta(x_1^t) &\triangleq \mathbb{P}_\theta(x_{t+1} = \cdot \mid x_1^t) \\ &= (\text{logit}_t / \|\text{logit}_t\|_1) \in [0, 1]^2, \quad (\text{Prediction}) \end{aligned}$$

where the MambaZero block is

$$\begin{aligned} H_t &= a_t H_{t-1} + \tilde{\mathbf{x}}_t \mathbf{b}_t^\top \in \mathbb{R}^{ed \times N}, \\ \mathbf{y}_t &= H_t \mathbf{c}_t \in \mathbb{R}^{ed}, \\ \mathbf{o}_t &= W_o \mathbf{y}_t \in \mathbb{R}^d, \end{aligned} \quad (\text{MambaZero})$$

and the input-selective terms $a_t, \tilde{\mathbf{x}}_t, \mathbf{b}_t$ and $\mathbf{c}_t$ are computed as in Eq. (2) without ReLU and just the convolution. Let $\theta = (\mathbf{e}_0, \mathbf{e}_1, \theta_{\text{MambaZero}}, W_\ell) \in \mathbb{R}^D$ denote the full set of parameters for appropriate $D \geq 1$.

4.2. Main theorem for first-order Markov

Towards establishing how MambaZero can represent Laplacian smoothing, we start with the random first-order Markov sequences. Our main result is the following.

Theorem 1 (MambaZero represents order-1 Laplacian smoothing). *For the canonical MambaZero model with dimensions $d = N = 2$, $e = 1$, and convolution window $w = 2$, there is a choice of parameters such that the model prediction is arbitrarily close to the Laplacian estimator for random first-order Markov chains. More formally, for any $\beta > 0$ and $\epsilon \in (0, 1)$, there exists a set of parameters θ such that, for all sequences $(x_t)_{t \geq 1}$ and all $t \geq 1$,*

$$D_{\text{KL}}\left(\mathbb{P}_\beta^{(1)}(\cdot \mid x_1^t) \parallel \mathbb{P}_\theta(\cdot \mid x_1^t)\right) \leq \epsilon.$$

Remark. The KL divergence above is precisely the penalty paid in the cross-entropy loss at time t (Eq. (3)) when using the predictor $\mathbb{P}_\theta$ instead of the optimal $\mathbb{P}_\beta^{(1)}$. In other words, the theorem implies that the loss of MambaZero can be made arbitrarily close to that of the optimal.

We defer the full proof to § B and outline the sketch below.

4.2.1. PROOF SKETCH

Main idea. To build our intuition towards how MambaZero can realize the add- β counting estimator for first-order Markov sequences, let’s focus on the core *MambaZero* block. The key observation here is the following: if the state H_{t-1} can capture all the transition counts $i \rightarrow j$ till x_1^{t-1} , the new state H_t can be updated to account for the current transition $x_{t-1} \rightarrow x_t$ on top of the existing counts, by a suitable choice of $a_t, \tilde{\mathbf{x}}_t$, and $\mathbf{b}_t$. Then the relevant count information corresponding to the current prefix x_t could be read off from the state projection $\mathbf{y}_t = H_t \mathbf{c}_t$, and be modified to account for β -smoothing via the *Linear* and *Prediction* layers. Buttressing this idea are two key empirical facts, which in fact hold for any $k \geq 1$, underpinning our construction:

(i) **State-to-state transition factor** $a_t \approx 1$ for all $t \geq 1$. We empirically observe that when MambaZero model is trained on random first-order Markov data, at convergence we have $a_t \approx 1$ for all $t \geq 1$ (Fig. 4). Since a_t modulates how much past information flows into the present, $a_t = 1$ serves as a natural choice when the states H_t carry count information till current tokens, which we would like to update without any diminishing factor. Note that this can be easily achieved by setting either a or Δ_t to be zero in Eq. (2), which we empirically observe as well.

(ii) **Convolution window** $w \geq k + 1$. Recalling that k is the Markov order, we empirically observe that the window that $w = k + 1$ is sufficient for the full Mamba to learn the Laplacian smoothing on k^{th} -order Markov chains. To

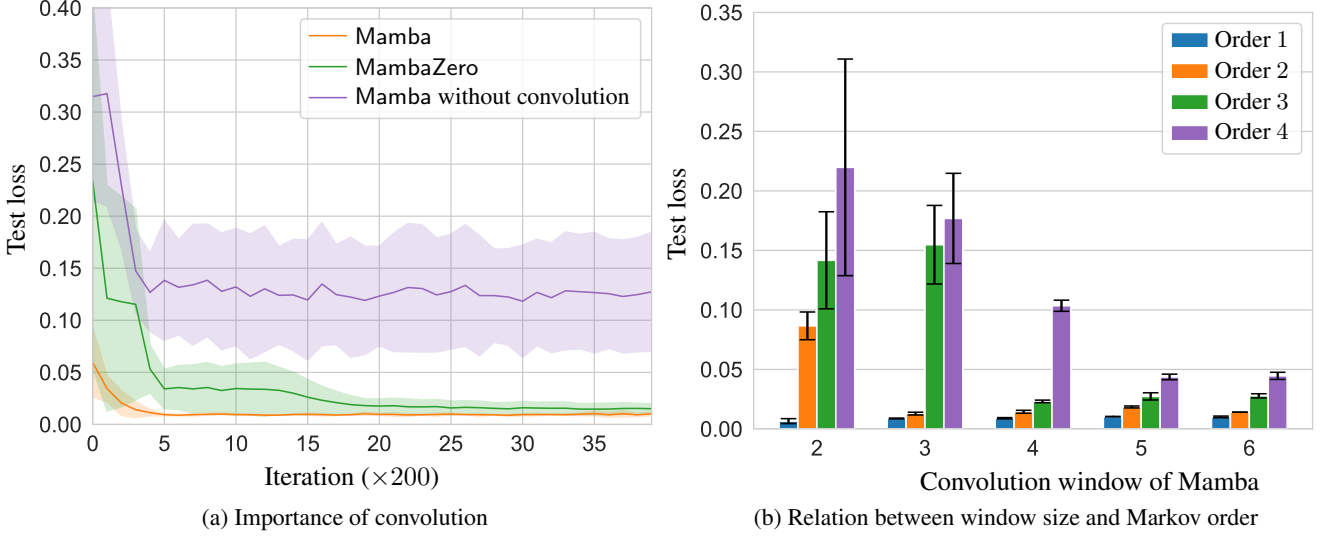


Figure 3. (a) illustrates the fundamental role of convolution, without which the model fails to learn the task. In contrast, a simplified variant with just the convolution (MambaZero) matches the performance of that of the full model. (b) highlights the relation between the Markov order k and the window size w of Mamba. It is required that $w \geq k + 1$ for the model to learn the order- k prediction task.

understand why, note that in the MambaZero architecture above, apart from the MambaZero block, all remaining equations operate on the current token at time t . In the MambaZero block, same as the Mamba block except ReLU, the dependency of the output y_t on the previous tokens is due to that of the state H_t on $(\tilde{x}_t, b_t)$ in the update equation, and of c_t in the state projection. Since $(\tilde{x}_t, b_t, c_t)$ depend on the past through the convolutions, a window of size $k + 1$ enables them to keep track of the current token as well as its length- k prefix, which is necessary to compute the counts needed in Laplacian-smoothing. On the other hand, if $w_X, w_B \leq k$, then one can find *confusable* sequences, i.e. sequences that share the same number of occurrences of all length- k prefixes, but whose counts of the tokens following each prefix is different, resulting in the model’s estimate to deviate from that of the optimal add- β . We refer to § B.1 for more details. While having all the window sizes $w_X, w_B, w_c \geq k + 1$ is sufficient, it can be further strengthened to $w_c = k$ (§ B.1).

We now detail our construction for the order-1 case, capitalizing on these insights.

Construction. Let us fix $w = k + 1 = 2$. Then, $\tilde{x}_t$ and b_t only depend on the current token x_t and the previous one x_{t-1} , while c_t only depends on x_t . Thus, $\tilde{x}_t$ and b_t can only take four possible values depending on the last transition in the sequence, whereas c_t only two. To ease the notation, we will denote these values by $\tilde{x}^{(ij)}$, $b^{(ij)}$, and $C^{(i)}$ respectively, for $i, j \in \{0, 1\}$. Additionally, at $t = 1$, these terms depend only on the current symbol, taking two additional values each, denoted by $\tilde{x}^{(i)}$, $b^{(i)}$. Let n_{ij} denote

the number of transitions $i \rightarrow j$ in the input sequence x_1^t . Then, unfolding the state update recursion in MambaZero,

$$H_t = \tilde{x}_0 b_0^\top + \sum_{ij} n_{ij} \tilde{x}^{(ij)} b^{(ij)\top}. \quad (4)$$

Thus the output of the MambaZero block is

$$o_t = W_o \tilde{x}_0 b_0^\top c_t + \sum_{ij} n_{ij} W_o \tilde{x}^{(ij)} b^{(ij)\top} c_t. \quad (5)$$

While the output in Eq. (5) depends on all the transition counts, in view of Laplacian-smoothing, we ideally want only those counts pertaining to relevant transitions, i.e. if $x_t = 0$, the counts n_{00} and n_{01} , and vice versa. To this end, we empirically observe that at convergence, the model’s parameters are such that

$$b^{(11)\top} c^{(0)} \approx 0 \text{ and } b^{(00)\top} c^{(1)} \approx 0. \quad (6)$$

Due to this property, the counts n_{00} and n_{11} do not contribute to the output o_t , respectively when $x_t = 1$ and $x_t = 0$. On the other hand, another key theoretical insight is that the transition counts in binary sequences are strongly correlated. Specifically, n_{01} and n_{10} are at most one apart: since, every time a transition $0 \rightarrow 1$ occurs, either the sequence is followed by only 1’s until the end, or a subsequent transition $1 \rightarrow 0$ also occurs, and vice versa. In fact, the precise relationship depends on the initial token x_1 , and on the current token x_t . If $x_1 = x_t$, then $n_{01} = n_{10}$; if $x_1 = 0$ and $x_t = 1$, then $n_{01} = n_{10} + 1$; while if $x_1 = 1$ and $x_t = 0$, then $n_{10} = n_{01} + 1$. Therefore, the dependency of the output on n_{01} is in fact a dependency on n_{10} , and

vice versa. As we will see, we can leverage this property to help MambaZero realize Laplacian smoothing with just two-dimensional embeddings, i.e. $d = 2$. Stitching these facts, the final logits in the **Linear** layer depend on the first and current token via

$$\begin{aligned} \text{logit}_t = & W_\ell \mathbf{x}_t + W_\ell W_o \tilde{\mathbf{x}}^{(x_1)} \mathbf{b}^{(x_1)\top} \mathbf{c}^{(x_t)} \\ & + \mathbb{1}_{\{x_1 \neq x_t\}} W_\ell W_o \tilde{\mathbf{x}}^{(x_1 x_t)} \mathbf{b}^{(x_1 x_t)\top} \mathbf{c}^{(x_t)} \\ & + \sum_j n_{x_t j} W_\ell W_o \tilde{\mathbf{x}}^{(x_t j)} \mathbf{b}^{(x_t j)\top} \mathbf{c}_t. \end{aligned} \quad (7)$$

The final step is to then show that for properly chosen parameters, one can make the two vectors associated with the counts, i.e. $W_\ell W_o \tilde{\mathbf{x}}^{(x_t j)} \mathbf{b}^{(x_t j)\top} \mathbf{c}_t$, for $j \in \{0, 1\}$, to be orthogonal, and the other vectors, independent of the counts, to sum up to a vector close to $(\beta, \beta)^\top$. Subsequently, the L_1 normalization in **Prediction** layer will give a next-token probability estimate, matching that of the add- β estimator. We refer to § B.2 for full details.

4.3. Higher order processes ($k > 1$)

While empirical evidence suggests that Thm. 1 would also hold for $k > 1$ (Figs. 1b, 3b), theoretically the proof becomes intractable due to the difficulty in tracking the correlations between the transition counts as k increases. Nonetheless, we believe that it is possible to extend the main proof ideas from the first-order case to here, though with a more involved construction, leading to the conjecture:

Conjecture 1. *The canonical MambaZero model can implement a predictor that is arbitrarily close to the add- β estimator, for the binary order- k Markov setting, with dimensions $d = N = 2^{k+1}$, $e = 1$, and window $w = k + 1$.*

We further strengthen this claim by the following matching lower bound, which shows that, with finite bit precision, any recurrent architecture such as Mamba cannot implement the Laplacian estimator with arbitrarily small error if the hidden dimension does not scale as $\Omega(2^k)$.

Theorem 2. *Consider a recurrent model of the form*

$$\begin{aligned} H_t &= h(H_{t-1}, x_t), \\ y_t &= \mathbb{P}_\theta(\cdot | x_1^t) = g(H_t), \end{aligned}$$

with transformations (h, g) , where $H_t \in \mathbb{R}^d$ and the model has a bit precision of p . Suppose that the Markov kernel P is sampled from the Dirichlet prior with $\beta = 1$, $P \sim \text{Dir}(1 \cdot 1)$. Suppose also that the recurrent architecture satisfies the following point-wise guarantee: for any sufficiently large t , almost surely over P and $x_1^t \sim P$,

$$\left\| \mathbb{P}_\theta(\cdot | x_1^t) - \mathbb{P}_1^{(k)}(\cdot | x_1^t) \right\|_\infty \leq \varepsilon. \quad (8)$$

Then, the recurrent architecture must satisfy,

$$d \cdot p \geq 2^k (1 - 3\varepsilon) \log(1/\varepsilon).$$

Remark. While we used time-invariant functions (h, g) in Thm. 2 for notational simplicity, the proof does not rely on this fact and holds for any general $H_t = h_t(H_{t-1}, x_t)$ and $y_t = g_t(H_t)$, which subsumes Mamba.

We defer the proof and additional details to App. C.

5. Beyond Markov

5.1. Switching Markov model

A key component of **Mamba** enabling selectivity is the state-transition factor a_t , that controls the flow of information from the past state H_{t-1} to the current H_t : if $a_t = 1$, the past information is fully utilized in computing the current state, and hence the output, whereas $a_t = 0$ completely ignores the past. In the Markovian setting considered so far, the role of a_t has largely been dormant: $a_t \approx 1$ for all $t \geq 1$, as the optimal Laplacian predictor requires counts of all transitions, demanding the use of full past (Sec. 4.2). To better highlight this selectivity mechanism, we consider a non-Markovian process, where the role of a_t becomes fundamental. Specifically, we focus on the *switching Markov process*, where we add a *switch* token to the binary alphabet, i.e. we consider $\mathcal{X} = \{0, 1, S\}$. The key difference here compared to the random Markov generation in Sec. 2.1 is that until we hit switch token, we follow the same binary Markov sequence generation as the former, but once the switch state is reached, we sample a new Markov kernel and then generate a new Markov sequence. The switch tokens are sampled according to a parallel i.i.d. Bernoulli process with probability p_{switch} (0.01 in our experiments). The sampling process is described in detail in § D.2.

Mamba learns the optimal estimator. With this data model, the optimal prediction strategy is to use the add- β estimator in between two switch tokens, and reset the transition counts every time a switch occurs. We provide empirical evidence in § D.2. Indeed, Fig. 5 illustrates that Mamba implements precisely this strategy, closely tracking the switching events via the transition factor a_t : it sets a_t to be zero whenever $x_t = S$ and to one otherwise. This enables the model to zero out the transition counts at every switch event, so that it can estimate the statistics of the new chain from scratch.

5.2. Natural language modeling

To test the generality of our finding that convolution plays key role on Markovian data (Fig. 3), we conduct experiments on the language modeling task using the WikiText-103 dataset with a sequence length of 256. We use a standard 2-layer Transformer consisting of attention and MLP modules with a model dimension of 256. To ensure a comparable parameter count, we stack the Mamba-2 cell across four layers, following the design in (Dao & Gu, 2024). By

adding or removing convolution in both these models, we obtain the results shown in Table 1. The results illustrate that while convolution significantly enhances the performance of Mamba-2, reducing perplexity from 30.68 to 27.55, the improvement for the Transformer is marginal. This highlights the fundamental role of convolution even on the complex language modeling tasks.

Table 1. Perplexity results on the WikiText-103 dataset for both models. (w/o conv) denotes the absence of convolution, while (w/ conv) indicates its use.

Model	# Params.	Perplexity ($\downarrow$)
Mamba-2 (w/o conv)	14.53	30.68
Mamba-2 (w/conv)	14.54	27.55
Transformer (w/o conv)	14.46	29.28
Transformer (w/ conv)	14.46	28.67

6. Related Work

State Space Models (Gu et al., 2020; 2021) have been recently introduced as an alternative recurrent architecture aimed at rivaling the well established transformer backbone (Vaswani et al., 2017) widely used across a range of domains, from language modeling to vision. The model was originally introduced as a discretization of a time-continuous linear dynamical system (Gu et al., 2021). Recent works tried to re-frame the architecture from a linear recurrent perspective. For example (Orvieto et al., 2023b) tried to understand what components of the architecture are necessary, highlighting the importance of linearity, as well as of the particular parametrization that allows control over the stability of model.

However there are still many gaps in understanding this family of models (Team et al., 2024), such as questions around expressivity (Orvieto et al., 2023a). This is particularly important given the proliferation of Mamba-inspired architectures that has emerged since the introduction of the model. For example, Mamba-Spike (Qin & Liu, 2024) integrates a spiking front-end with the Mamba backbone, leveraging event-driven processing for efficient temporal data handling. The Mixture of Mamba approach combines Mamba’s selective state spaces with the Transformer’s Mixture of Experts (MoE) layer (Csordás et al., 2024), potentially offering faster convergence and improved performance scaling. Vision Mamba (Vim) (Zhu et al.) adapts the architecture for visual tasks, employing bidirectional Mamba blocks to process high-resolution images efficiently. Besides specific adaptation of the model into new architectures, the core Mamba block also evolved, for example, by the introduction of gating in Mamba 2 (Gu & Dao, 2023b) and similar SSM architectures (De et al., 2024; Beck et al., 2024) which for

example invalidates to a certain extent the convolutional view of the architecture.

Our work squarely focuses on understanding the representation power of Mamba, and ICL, which remains an unanswered question. We believe such insights can help navigate the many flavors of SSM architectures.

In this space, recent studies have shown that SSMs can perform gradient-based learning for in-context adaptation, similar to transformers (Sushma et al., 2024), where they show that a single SSM layer, augmented with local self-attention, is able to reproduce outputs of an implicit linear model after one step of gradient descent. While some work suggests that Mamba’s ICL abilities rival state-of-the-art transformer models (Grazzi et al., 2024), other research indicates that the pretrained Mamba models perform worse in-context learning improvements than comparable transformer models (Halloran et al., 2024; Akyürek et al., 2024). SSMs have also demonstrated promising results in in-context reinforcement learning tasks, offering fast and performant solutions (Lu et al., 2024). Joseph et al. (2024) has introduced a novel weight construction for SSMs, enabling them to predict the next state of any dynamical system in context. These findings highlight the potential of SSMs as efficient alternatives to transformers for ICL tasks, particularly those involving long input sequences.

7. Conclusion

Structured state space sequence models (SSMs) and Selective SSMs such as Mamba have shown remarkable inference speed ups over transformers while achieving comparable or superior performance on complex language modeling tasks. In this paper, we studied in-context learning (ICL) capabilities of Mamba on Markov chains and show that, unlike transformers, even a single-layer Mamba efficiently learns the in-context Laplacian smoothing estimator. To explain this, we theoretically characterized the representation capacity of Mamba and reveal the fundamental role of convolution in enabling it. We provided empirical results that align with these theoretical insights. Extending our results to deeper Mamba models and understanding the role of depth are some interesting future directions.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Akyürek, E., Wang, B., Kim, Y., and Andreas, J. In-context language learning: Architectures and algorithms. *arXiv preprint arXiv:2401.12973*, 2024.
- Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., and Hochreiter, S. xlstm: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*, 2024.
- Cesa-Bianchi, N. and Lugosi, G. *Prediction, Learning, and Games*. Cambridge University Press, 2006.
- Cirone, N. M., Orvieto, A., Walker, B., Salvi, C., and Lyons, T. Theoretical foundations of deep selective state-space models, 2025. URL <https://arxiv.org/abs/2402.19047>.
- Csordás, R., Irie, K., Schmidhuber, J., Potts, C., and Manning, C. D. Moeut: Mixture-of-experts universal transformers. *arXiv preprint arXiv:2405.16039*, 2024.
- Dao, T. and Gu, A. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. *arXiv preprint arXiv:2405.21060*, 2024.
- De, S., Smith, S. L., Fernando, A., Botev, A., Cristian-Muraru, G., Gu, A., Haroun, R., Berrada, L., Chen, Y., Srinivasan, S., et al. Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models. *arXiv preprint arXiv:2402.19427*, 2024.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding, 2018. URL <https://arxiv.org/abs/1810.04805>.
- Edelman, B. L., Edelman, E., Goel, S., Malach, E., and Tsilivis, N. The Evolution of Statistical Induction Heads: In-Context Learning Markov Chains, 2024.
- Grazzi, R., Siems, J., Schrod, S., Brox, T., and Hutter, F. Is mamba capable of in-context learning? *arXiv preprint arXiv:2402.03170*, 2024.
- Gu, A. and Dao, T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv preprint arXiv:2312.00752*, 2023a.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023b.
- Gu, A., Dao, T., Ermon, S., Rudra, A., and Ré, C. Hippo: Recurrent memory with optimal polynomial projections. In *Advances in Neural Information Processing Systems*, volume 33, pp. 1474–1487, 2020.
- Gu, A., Johnson, I., Goel, K., Saab, K., Dao, T., Rudra, A., and Ré, C. Combining recurrent, convolutional, and continuous-time models with linear state space layers. In *Advances in Neural Information Processing Systems*, volume 34, pp. 572–585, 2021.
- Halloran, J. T., Gulati, M., and Roysdon, P. F. Mamba state-space models can be strong downstream learners. *arXiv preprint arXiv:2406.00209*, 2024.
- Joseph, F. A., Haefeli, K. K., Liniger, N., and Gulcehre, C. Hippo-prophecy: State-space models can provably learn dynamical systems in context. *arXiv preprint arXiv:2407.09375*, 2024.
- Kingma, D. and Ba, J. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Laplace, P. S. *Essai philosophique sur les probabilités*. Courcier, Paris, France, 1814. Reprinted by Cambridge University Press, 2009. In the reprint, the estimator appears on page 23.
- Lu, C., Schroecker, Y., Gu, A., Parisotto, E., Foerster, J., Singh, S., and Behbahani, F. Structured state space models for in-context reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.
- Merhav, N. and Feder, M. Universal prediction. *IEEE Transactions on Information Theory*, 44(6):2124–2147, 1998. doi: 10.1109/18.720534.
- Nichani, E., Damian, A., and Lee, J. D. How Transformers Learn Causal Structure with Gradient Descent. *arXiv preprint arXiv:2402.14735*, 2024.
- Orvieto, A., De, S., Gulcehre, C., Pascanu, R., and Smith, S. L. On the universality of linear recurrences followed by nonlinear projections. *arXiv preprint arXiv:2307.11888*, 2023a.
- Orvieto, A., Smith, S. L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., and De, S. Resurrecting recurrent neural networks for long sequences. *arXiv preprint arXiv:2303.06349*, 2023b.
- Pagliardini, M. GPT-2 modular codebase implementation. <https://github.com/epfml/llm-baselines>, 2023.
- Qin, J. and Liu, F. Mamba-spike: Enhancing the mamba architecture with a spiking front-end for efficient temporal data processing. *arXiv preprint arXiv:2408.11823*, 2024.
- Radford, A. and Narasimhan, K. Improving language understanding by generative pre-training. 2018. URL <https://api.semanticscholar.org/CorpusID:49313245>.

- Rajaraman, N., Bondaschi, M., Ramchandran, K., Gastpar, M., and Makkuva, A. V. Transformers on markov data: Constant depth suffices, 2024. URL <https://arxiv.org/abs/2407.17686>.
- Rissanen, J. Universal coding, information, prediction, and estimation. *IEEE Transactions on Information Theory*, 30(4):629–636, 1984. doi: 10.1109/TIT.1984.1056936.
- Sanford, C., Hsu, D., and Telgarsky, M. One-layer transformers fail to solve the induction heads task, 2024. URL <https://arxiv.org/abs/2408.14332>.
- Sushma, N. M., Tian, Y., Mestha, H., Colombo, N., Kappel, D., and Subramoney, A. State-space models can learn in-context by gradient descent. *arXiv preprint arXiv:2410.11687*, 2024.
- Team, J., Lenz, B., Araz, A., Bergman, A., Manevich, A., Peleg, B., Aviram, B., Almagor, C., Fridman, C., Padnos, D., et al. Jamba-1.5: Hybrid transformer-mamba models at scale. *arXiv preprint arXiv:2408.12570*, 2024.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, pp. 5998–6008, 2017.
- Zhu, L., Liao, B., Zhang, Q., Wang, X., Liu, W., and Wang, X. Vision mamba: Efficient visual representation learning with bidirectional state space model. arxiv 2024. *arXiv preprint arXiv:2401.09417*.

A. Preliminaries about Laplacian smoothing

Laplacian smoothing is a mature and well understood topic. An account can be found, e.g., in (Merhav & Feder, 1998; Cesa-Bianchi & Lugosi, 2006). We provide a brief outline of how Laplacian smoothing applies in our context. Consider a given sequence $(x_t)_{t=1}^T$. For every length- k prefix $i_1^k \in \mathcal{X}^k$, let $(x_t)_{i_1^k}$ be the subsequence of tokens preceded by i_1^k . Note that, since each sequence (x_t) is generated by a k -th order Markov chain, all the tokens in the sequence with the same length- k prefix share the same conditional probability distribution. Furthermore, since each of the conditional distributions of the chain is randomly chosen independently from the others, the subsequence $(x_t)_{i_1^k}$ is a sufficient statistic to estimate the probability distribution of all the tokens with the same prefix i_1^k . Therefore, the optimal prediction for a sequence $(x_t)_{t=1}^T$ is given by employing the optimal predictor for each i.i.d. subsequence $(x_t)_{i_1^k}$, for every $i_1^k \in \mathcal{X}^k$. Since each conditional distribution is sampled from a Dirichlet distribution with parameter β , it is well known that the optimal predictor for such subsequences is the add-constant estimator, with constant equal to β . More specifically, if $x_{t-k}^{t-1} = i_1^k$, then the optimal estimation for x_t is

$$\mathbb{P}_\beta^{(k)}(x_t = j \mid x_{t-1}^{t-1}) = \frac{n_j + \beta}{n + |\mathcal{X}|\beta}, \quad (9)$$

where n_j is the number of times token j appears in the subsequence $(x_t)_{i_1^k}$, and n is the length of the subsequence. It is important to note that, to be able to implement such a predictor requires in-context capabilities: at inference, in order to optimally predict the next token, the model must be able to look into the previous tokens of the test sequence, and count the tokens with the correct prefix.

B. Preliminaries and Proof of Thm. 1

B.1. Empirical insights

Here we expand upon our empirical observations in 4.2.1, which form the basis of our proof.

State-to-state transition factor $a_t \approx 1$ for all $t \geq 1$. We empirical evidence supporting this observation in Fig. 4.

Convolution window $w \geq k + 1$. Recalling that k is the Markov order, we empirically observe that the window that $w = k + 1$ is sufficient for the full Mamba to learn the Laplacian smoothing on k^{th} -order Markov chains. To understand why, note that in the MambaZero architecture above, apart from the MambaZero block, all remaining equations operate on the current token at time t . In the MambaZero block, same as the Mamba block except ReLU, the dependency of the output y_t on the previous tokens is due to that of the state H_t on $(\tilde{x}_t, b_t)$ in the update equation, and of c_t in the state projection. Since $(\tilde{x}_t, b_t, c_t)$ depend on the past through the convolutions, a window of size $k + 1$ enables them to keep track of the current token as well as its length- k prefix, which is necessary to compute the counts needed in Laplacian-smoothing. On the other hand, if $w \leq k$, then one can find *confusable* sequences, i.e. sequences that share the same number of occurrences of all length- k prefixes, but whose counts of the tokens following each prefix is different.

For such sequences, the state H_t is the same, and so are the predicted probabilities by the Mamba model; however, the optimal estimator, depending on the transition counts, would give very different probability estimates, allowing Mamba’s prediction loss to deviate from that of the optimal. For example, consider $k = 1$. If $w = 1$, then $(\tilde{x}_t, b_t, c_t)$ depend only on the current token x_t . Then, consider the two sequences $x = (0, 1, 0, 1, 0, 1)$ and $\tilde{x} = (0, 0, 0, 1, 1, 1)$. At time $t = 6$, these two sequences would give the same state H_t and the same output y_t , since they share the same number of tokens 0 and 1. Therefore, the estimated probability given by the model would be the same in both cases. However, the optimal add-constant estimator (with $\beta = 1$) would estimate the probability of $x_{t+1} = 1$ to be $1/4$ for x , and $3/4$ for $\tilde{x}$.

Further, it is sufficient that the convolution for c_t has window $w_C = k$. That is, the convolution conv_C involved in the computation of c_t can have a window size equal to the Markov order k (i.e., one less than conv_X and conv_B) without affecting the model’s capability of learning the task (or, equivalently, the left-most kernel coefficients of conv_C can be taken to be zero). Intuitively, this is because the role of c_t in the state projection is to select the correct transition counts for the computation of the estimator, distilled into y_t . In order to do so, it is sufficient to know the length- k context of the current symbol x_t , which can be encoded by a convolution with window size k .

B.2. Proof of Thm. 1

Fix $\epsilon > 0$ and let $\beta > 0$ be the constant of the considered add-constant estimator. Let us fix $a = 0$ and $\Delta_t = 1$, so that $a_t = 1$, for all $t \geq 1$. This can be done by picking, e.g., $w_\Delta = 0$ and δ such that $\text{softplus}(\delta) = 1$. Let us compactly denote

the convolution kernels as

$$\text{conv}_X = \begin{pmatrix} \alpha_{00} & \alpha_{01} \\ \alpha_{10} & \alpha_{11} \end{pmatrix}, \quad \text{conv}_B = \begin{pmatrix} \gamma_{00} & \gamma_{01} \\ \gamma_{10} & \gamma_{11} \end{pmatrix} \quad (10)$$

where each row corresponds to the kernel weights applied time-wise to each coordinate of the input sequence $(\mathbf{x}_t)_{t \geq 1}$. Since the window for conv_C is $w_C = 1$, we can simply assume w.l.o.g. that $C_t = W_C \mathbf{x}_t$.

Let us denote the embedding vectors to be $\mathbf{e}_0 = (e_{00}, e_{01})^\top$ and $\mathbf{e}_1 = (e_{10}, e_{11})^\top$, and assume that the vectors are not collinear. Take also $W_X = W_B$ such that

$$W_X \mathbf{e}_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad W_X \mathbf{e}_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (11)$$

and take W_C such that

$$W_C \mathbf{e}_0 = \begin{pmatrix} c_0 \\ 0 \end{pmatrix}, \quad W_C \mathbf{e}_1 = \begin{pmatrix} 0 \\ c_1 \end{pmatrix}. \quad (12)$$

Let us also take the kernels of conv_X and conv_B to be the same across coordinates, i.e.,

$$\text{conv}_X = \begin{pmatrix} \alpha_0 & \alpha_1 \\ \alpha_0 & \alpha_1 \end{pmatrix}, \quad \text{conv}_B = \begin{pmatrix} \gamma_0 & \gamma_1 \\ \gamma_0 & \gamma_1 \end{pmatrix} \quad (13)$$

such that the following conditions are satisfied:

$$\begin{cases} \alpha_0 \gamma_0 + \alpha_1 \gamma_1 = 0 \\ \alpha_0 \gamma_1 + \alpha_1 \gamma_0 > 0 \\ \alpha_0 \neq \alpha_1 \\ \frac{\alpha_0 \gamma_1}{\alpha_0 \gamma_1 + \alpha_1 \gamma_0} = -\beta \epsilon \end{cases} \quad (14)$$

Note that, with such a choice of parameters, we have

$$X^{(0)} = \begin{pmatrix} \alpha_1 \\ \alpha_0 \end{pmatrix}, \quad X^{(1)} = \begin{pmatrix} 0 \\ \alpha_1 \end{pmatrix}, \quad B^{(0)} = \begin{pmatrix} \gamma_1 \\ \gamma_0 \end{pmatrix}, \quad B^{(1)} = \begin{pmatrix} 0 \\ \gamma_1 \end{pmatrix} \quad (15)$$

$$C^{(0)} = \begin{pmatrix} c_0 \\ 0 \end{pmatrix}, \quad C^{(1)} = \begin{pmatrix} 0 \\ c_1 \end{pmatrix} \quad (16)$$

$$X^{(00)} = \begin{pmatrix} \alpha_0 + \alpha_1 \\ 0 \end{pmatrix}, \quad X^{(01)} = \begin{pmatrix} \alpha_0 \\ \alpha_1 \end{pmatrix}, \quad X^{(10)} = \begin{pmatrix} \alpha_1 \\ \alpha_0 \end{pmatrix}, \quad X^{(11)} = \begin{pmatrix} 0 \\ \alpha_0 + \alpha_1 \end{pmatrix} \quad (17)$$

$$B^{(00)} = \begin{pmatrix} \gamma_0 + \gamma_1 \\ 0 \end{pmatrix}, \quad B^{(01)} = \begin{pmatrix} \gamma_0 \\ \gamma_1 \end{pmatrix}, \quad B^{(10)} = \begin{pmatrix} \gamma_1 \\ \gamma_0 \end{pmatrix}, \quad B^{(11)} = \begin{pmatrix} 0 \\ \gamma_0 + \gamma_1 \end{pmatrix}. \quad (18)$$

(We replaced the vector notation of Sec. 4 with matrix notation, so that $X^{(0)}$ has to be intended as $\tilde{\mathbf{x}}^{(0)}$, and so on.) Take also $W_o = W_\ell = I$. With this choice of parameters, the final logit vector becomes

$$\begin{aligned} \text{logit}_t &= W_\ell \mathbf{x}_t + W_\ell W_o X^{(x_1)} B^{(x_1)\top} C^{(x_t)} + \mathbb{1}_{\{x_1 \neq x_t\}} W_\ell W_o X^{(x_1 x_t)} B^{(x_1 x_t)\top} C^{(x_t)} \\ &\quad + \sum_j n_{x_t j} W_\ell W_o X^{(x_t j)} B^{(x_t j)\top} C_t \end{aligned} \quad (19)$$

$$= \begin{pmatrix} e_{00} + c_0 \alpha_1 \gamma_1 \\ e_{01} \end{pmatrix} + \mathbb{1}_{\{x_1=1\}} \cdot \begin{pmatrix} 0 \\ c_0 \alpha_0 \gamma_1 \end{pmatrix} + n_{00} \cdot \begin{pmatrix} (\alpha_0 + \alpha_1)(\gamma_0 + \gamma_1) c_0 \\ 0 \end{pmatrix} + n_{01} \cdot \begin{pmatrix} (\alpha_0 \gamma_0 + \alpha_1 \gamma_1) c_0 \\ (\alpha_0 \gamma_1 + \alpha_1 \gamma_0) c_0 \end{pmatrix} \quad (20)$$

$$= \begin{pmatrix} e_{00} + c_0 \alpha_1 \gamma_1 \\ e_{01} \end{pmatrix} + \mathbb{1}_{\{x_1=1\}} \cdot \begin{pmatrix} 0 \\ c_0 \alpha_0 \gamma_1 \end{pmatrix} + n_{00} \cdot \begin{pmatrix} (\alpha_0 \gamma_1 + \alpha_1 \gamma_0) c_0 \\ 0 \end{pmatrix} + n_{01} \cdot \begin{pmatrix} 0 \\ (\alpha_0 \gamma_1 + \alpha_1 \gamma_0) c_0 \end{pmatrix} \quad (21)$$

if $x_t = 0$, and

$$\text{logit}_t = \begin{pmatrix} e_{10} \\ e_{11} + c_1 \alpha_1 \gamma_1 \end{pmatrix} + \mathbb{1}_{\{x_1=0\}} \cdot \begin{pmatrix} c_1 \alpha_0 \gamma_1 \\ 0 \end{pmatrix} + n_{10} \cdot \begin{pmatrix} (\alpha_0 \gamma_1 + \alpha_1 \gamma_0) c_1 \\ (\alpha_0 \gamma_0 + \alpha_1 \gamma_1) c_1 \end{pmatrix} + n_{11} \cdot \begin{pmatrix} 0 \\ (\alpha_0 + \alpha_1)(\gamma_0 + \gamma_1) c_1 \end{pmatrix} \quad (22)$$

$$= \begin{pmatrix} e_{10} \\ e_{11} + c_1 \alpha_1 \gamma_1 \end{pmatrix} + \mathbb{1}_{\{x_1=0\}} \cdot \begin{pmatrix} c_1 \alpha_0 \gamma_1 \\ 0 \end{pmatrix} + n_{10} \cdot \begin{pmatrix} (\alpha_0 \gamma_1 + \alpha_1 \gamma_0) c_1 \\ 0 \end{pmatrix} + n_{11} \cdot \begin{pmatrix} 0 \\ (\alpha_0 \gamma_1 + \alpha_1 \gamma_0) c_1 \end{pmatrix} \quad (23)$$

if $x_t = 1$. Take now

$$e_{00} = e_{11} = (\alpha_0 \gamma_1 + \alpha_1 \gamma_0) \beta c_0 - \alpha_1 \gamma_1 c_0 \quad (24)$$

$$e_{01} = e_{10} = (\alpha_0 \gamma_1 + \alpha_1 \gamma_0) \beta c_0 - \alpha_0 \gamma_1 c_0 \quad (25)$$

With this choice of parameters, after the layer normalization, the final output probability vector is

$$f_{\theta}(x_1^t) = \left(\frac{n_{00} + \beta}{n_{00} + n_{01} + 2\beta + \mathbb{1}_{\{x_1=0\}} \cdot \beta \epsilon}, \frac{n_{01} + \beta + \mathbb{1}_{\{x_1=0\}} \cdot \beta \epsilon}{n_{00} + n_{01} + 2\beta + \mathbb{1}_{\{x_1=0\}} \cdot \beta \epsilon} \right)^{\top} \quad (26)$$

if $x_t = 0$, and

$$f_{\theta}(x_1^t) = \left(\frac{n_{10} + \beta + \mathbb{1}_{\{x_1=1\}} \cdot \beta \epsilon}{n_{10} + n_{11} + 2\beta + \mathbb{1}_{\{x_1=1\}} \cdot \beta \epsilon}, \frac{n_{11} + \beta}{n_{10} + n_{11} + 2\beta + \mathbb{1}_{\{x_1=1\}} \cdot \beta \epsilon} \right)^{\top} \quad (27)$$

if $x_t = 1$. Note that the resulting predicted probabilities exactly match the add- β estimator when $x_1 \neq x_t$, but they are slightly different when $x_1 = x_t$ due to the additional $\beta \epsilon$ factor. We now show that, when the additional factor is present, the two predictors nevertheless differ by at most ϵ in KL distance. We show it for the case $x_1 = x_t = 0$, the other case follows in the same way. In fact, note that

$$\frac{n_{01} + \beta + \beta \epsilon}{n_{00} + n_{01} + 2\beta + \beta \epsilon} = \frac{n_{01} + \beta}{n_{00} + n_{01} + 2\beta} \cdot \frac{1 + \frac{\beta \epsilon}{n_{01} + \beta}}{1 + \frac{\beta \epsilon}{n_{00} + n_{01} + 2\beta}}. \quad (28)$$

Now, since

$$1 \leq 1 + \frac{\beta \epsilon}{n_{01} + \beta} \leq 1 + \epsilon \quad (29)$$

and

$$1 \leq 1 + \frac{\beta \epsilon}{n_{00} + n_{01} + 2\beta} \leq 1 + \epsilon \quad (30)$$

we have that

$$\frac{n_{01} + \beta}{n_{00} + n_{01} + 2\beta} \leq \frac{n_{01} + \beta + \beta \epsilon}{n_{00} + n_{01} + 2\beta + \beta \epsilon} \cdot (1 + \epsilon) \quad (31)$$

but we also have

$$\frac{n_{00} + n_{01} + 2\beta + \beta \epsilon}{n_{00} + n_{01} + 2\beta} \leq 1 + \frac{\beta \epsilon}{n_{00} + n_{01} + 2\beta} \leq 1 + \epsilon, \quad (32)$$

so that

$$\begin{aligned} D_{\text{KL}} \left(\mathbb{P}_{\beta}^{(1)}(\cdot | x_1^t) \| \mathbb{P}_{\theta}(\cdot | x_1^t) \right) &= \mathbb{P}_{\beta}^{(1)}(x_{t+1} = 0 | x_1^t) \log \frac{\mathbb{P}_{\beta}^{(1)}(x_{t+1} = 0 | x_1^t)}{\mathbb{P}_{\theta}(x_{t+1} = 0 | x_1^t)} \\ &\quad + \mathbb{P}_{\beta}^{(1)}(x_{t+1} = 1 | x_1^t) \log \frac{\mathbb{P}_{\beta}^{(1)}(x_{t+1} = 1 | x_1^t)}{\mathbb{P}_{\theta}(x_{t+1} = 1 | x_1^t)} \end{aligned} \quad (33)$$

$$\begin{aligned} &= \frac{n_{00} + \beta}{n_{00} + n_{01} + 2\beta} \log \frac{\frac{n_{00} + \beta}{n_{00} + n_{01} + 2\beta}}{\frac{n_{00} + \beta}{n_{00} + n_{01} + 2\beta + \beta \epsilon}} \\ &\quad + \frac{n_{01} + \beta}{n_{00} + n_{01} + 2\beta} \log \frac{\frac{n_{01} + \beta}{n_{00} + n_{01} + 2\beta}}{\frac{n_{01} + \beta + \beta \epsilon}{n_{00} + n_{01} + 2\beta + \beta \epsilon}} \end{aligned} \quad (34)$$

$$\leq \frac{n_{00} + \beta}{n_{00} + n_{01} + 2\beta} \log(1 + \epsilon) + \frac{n_{01} + \beta}{n_{00} + n_{01} + 2\beta} \log(1 + \epsilon) \quad (35)$$

$$\leq \log(1 + \epsilon) \quad (36)$$

$$\leq \epsilon \quad (37)$$

concluding the proof. $\square$

C. Proof of Thm. 2

Consider a recurrent model of the form $H_t = h(H_{t-1}, x_t)$ and $y_t = g(H_t)$ for each $t \geq 1$ where $H_t \in \mathbb{R}^d$ and the model has a bit precision of p . In this proof, we will assume that the state space of the underlying Markov chain is $\{0, 1\}$. By the recurrent architecture, the predicted distribution over the next token x_{t+k+1} is of the form,

$$y_{t+k} = \mathbb{P}_\theta (x_{t+k+1} = z \mid x_1^{t+k}) = g(z, H_{t+k}). \quad (38)$$

Recall that the add-1 estimator is defined as,

$$\frac{n(z, x_{t+1}^{t+k}) + 1}{n(x_{t+1}^{t+k}) + 2}, \quad (39)$$

where $n(z_1^k) = \sum_{i=1}^{t+1} \mathbb{I}(x_i^{i+k-1} = z_1^k)$ indicates the number of times z_1^k appears in the sequence. This is the optimal estimator for sequences drawn from the product-Dirichlet prior: for every i_1^k , $P(\cdot \mid i_1^k) \sim \text{Dir}(1 \cdot \mathbf{1})$, which is the distribution we will assume for this proof. Fixing x_1^t , we can write the add-1 estimator more explicitly as a function of x_{t+1}^{t+k} as,

$$\mathbb{P}_1^{(k)} (x_{t+k+1} = z \mid x_1^t, x_{t+1}^{t+k} = z_1^k) = \frac{n(z_1^k, z) + 1}{n(z_1^k) + 2}. \quad (40)$$

Now, fixing x_1^t , correctness of the recurrent model means that, almost surely over P drawn from the prior, and $x_1^t \sim P$ and $z_1^k \sim P(\cdot \mid x_1^t)$,

$$g(x_{t+k+1} = 0, H_{t+k}) \in \mathbb{P}_1^{(k)} (x_{t+k+1} = 0 \mid x_1^t, x_{t+1}^{t+k} = z_1^k) + [-\varepsilon, \varepsilon]. \quad (41)$$

where H_{t+k} is a function of x_1^t and z_1^k . As $t \rightarrow \infty$, under the randomness of the draw of $x_1^t \sim P$, by the strong law of large numbers RHS converges almost surely to the conditional distribution under P , almost surely over the choice of P from the product-Dirichlet prior. Here we use the fact that for P drawn from the product-Dirichlet prior, $P(z \mid z_1^k) > 0$ almost surely, and so the resulting distributions are exponentially mixing and ergodic. Namely, for each $z_1^k \in \{0, 1\}^k$, almost surely over P drawn from the product-Dirichlet prior,

$$\Pr \left(\limsup_{t \rightarrow \infty} \left| \mathbb{P}_1^{(k)} (x_{t+k+1} = 0 \mid x_1^t, x_{t+1}^{t+k} = z_1^k) - P(0 \mid z_1^k) \right| > \gamma \right) = 0 \quad (42)$$

for any $\gamma > 0$. Therefore, a necessary condition to satisfy Equation (41) is, for each $z_1^k \in \mathcal{X}^k$,

$$g(x_{t+k+1} = 0, H_{t+k}) \in P(0 \mid z_1^k) + [-\varepsilon - \eta_P(t), \varepsilon + \eta_P(t)]. \quad (43)$$

for some $\eta_P(t)$, which is a function of P satisfying $\limsup_{t \rightarrow \infty} \eta_P(t) = 0$ almost surely over P drawn from the prior; note that H_{t+k} is implicitly a function of x_1^t and z_1^k . Divide the interval $[0, 1]$ into $1/\varepsilon$ disjoint intervals of size ε each. Recall that $P(\cdot \mid z_1^k) \sim \rho = \text{Dir}(1 \cdot \mathbf{1})$, which implies that the random variable $P(0 \mid z_1^k)$ for each fixed z_1^k (randomness is over P) is distributed as,

$$\Pr_\rho [P(0 \mid z_1^k) = \cdot \mid z_1^k] = \text{Unif}([0, 1]). \quad (44)$$

Consider the buckets $B_\varepsilon = \{[0, \varepsilon), [\varepsilon, 2\varepsilon), \dots, [1 - \varepsilon, 1]\}$. Define the function $\text{round}(p) : [0, 1] \rightarrow \{0, \dots, |B_\varepsilon| - 1\}$ to return the index of the bucket in B_ε such that p falls in that bucket.

Lemma 1. Consider any function $f(z_1^k) : \mathcal{X}^k \rightarrow \{0, \dots, |B_\varepsilon| - 1\}$ such that, pointwise,

$$|\text{round}(P(0 \mid z_1^k)) - f(z_1^k)| \leq r. \quad (45)$$

Then, when $P(0 \mid z_1^k) \stackrel{i.i.d.}{\sim} \text{Dir}(1 \cdot \mathbf{1})$,

$$H_{\text{Shannon}}(\{f(z_1^k) : z_1^k \in \{0, 1\}^k\}) \geq 2^k ((1 - 3\varepsilon) \log(1/\varepsilon) - \log(2r + 1)) \quad (46)$$

where the randomness is over the draw of P and H_{Shannon} is the discrete Shannon entropy.

Proof. Recall that $P(0|z_1^k) \stackrel{\text{i.i.d.}}{\sim} \text{Unif}([0, 1])$ across $z_1^k \in \mathcal{X}^k$. Then,

$$\Pr(\text{round}(P(0|z_1^k)) = j) = \Pr(P(0|z_1^k) \in [\varepsilon(j - 1/2), \varepsilon(j + 1/2))) \quad (47)$$

$$= \begin{cases} \varepsilon & \text{if } 1 \leq j \leq |B_\varepsilon| - 2, \\ 3\varepsilon/2 & \text{if } j = 0 \text{ or } j = |B_\varepsilon| - 1. \end{cases} \quad (48)$$

This implies that, by independence of the $P(0|z_1^k)$'s across $z_1^k \in \mathcal{X}^k$,

$$H_{\text{Shannon}}(\{P(0|z_1^k) : z_1^k \in \mathcal{X}^k\}) \geq |\mathcal{X}|^k (1 - 3\varepsilon) \log(1/\varepsilon). \quad (49)$$

Let $e(z_1^k)$ be the random variable $P(0|z_1^k) - f(z_1^k)$. $P(0|z_1^k)$ is a measurable function of $f(z_1^k)$ and $e(z_1^k)$, and therefore,

$$H_{\text{Shannon}}(\{f(z_1^k) : z_1^k \in \mathcal{X}^k\} \cup \{e(z_1^k) : z_1^k \in \mathcal{X}^k\}) \geq H_{\text{Shannon}}(\{P(0|z_1^k) : z_1^k \in \mathcal{X}^k\}) \quad (50)$$

Note that $e(z_1^k)$ is bounded in the rate $\{-r, \dots, r\}$ and can take at most $2r + 1$ values. Therefore, $H(\{e(z_1^k) : z_1^k \in \mathcal{X}^k\}) \leq |\mathcal{X}|^k \log(2r + 1)$. Since $H(A, B) \leq H(A) + H(B)$, we have that,

$$H_{\text{Shannon}}(\{f(z_1^k) : z_1^k \in \mathcal{X}^k\}) \geq H_{\text{Shannon}}(\{P(0|z_1^k) : z_1^k \in \mathcal{X}^k\}) - H(\{e(z_1^k) : z_1^k \in \mathcal{X}^k\}) \quad (51)$$

$$\geq |\mathcal{X}|^k ((1 - 3\varepsilon) \log(1/\varepsilon) - \log(2r + 1)). \quad (52)$$

□

Recall that we are guaranteed that $g(x_{t+k+1} = 0, H_{t+k}) \in P(0|z_1^k) + [-\varepsilon - \eta_P(t), \varepsilon + \eta_P(t)]$. This implies that the recurrent model is able to recover $\text{round}(p)$ for $p = P(0|z_1^k)$ up to an error of $r = \lceil \eta_P(t)/\varepsilon \rceil$ for each $z_1^k \in \mathcal{X}^k$ by computing $\text{round}(\hat{p})$ where $\hat{p} = g(x_{t+k+1} = 0, H_{t+k})$. Informally, this just means that $\hat{p}$ is likely to fall in a bucket close to p . In combination with Lemma 1, for $f(z_1^k) = g(x_{t+k+1} = 0, H_{t+k})$ we have that,

$$H_{\text{Shannon}}(\{g(x_{t+k+1} = 0, H_{t+k}) : z_1^k \in \{0, 1\}^k\}) \geq 2^k ((1 - 3\varepsilon) \log(1/\varepsilon) - \log(2\lceil \eta_P(t)/\varepsilon \rceil + 1)) \quad (53)$$

Note however, that $g(x_{t+k+1} = 0, H_{t+k})$ is a function of z_1^k implicitly, through H_{t+k} (which is also a function of x_1^t). Since the dimensionality of H_{t+k} is d and the model is implemented to p bits of precision,

$$H_{\text{Shannon}}(\{g(x_{t+k+1} = 0, H_{t+k}) : z_1^k \in \{0, 1\}^k\}) \leq H_{\text{Shannon}}(H_{t+k}) \leq dp \quad (54)$$

where all randomness here is induced by the random draw of the k^{th} -order Markov kernel P . Therefore, for the correctness guarantee Equation (41) to hold, we need,

$$dp \geq 2^k ((1 - 3\varepsilon) \log(1/\varepsilon) - \log(2\lceil \eta_P(t)/\varepsilon \rceil + 1)) \quad (55)$$

in the limit $t \rightarrow \infty$, and noting that $\limsup_{t \rightarrow \infty} \eta_P(t) = 0$ almost surely over P drawn from the prior, it is necessary that,

$$dp \geq 2^k (1 - 3\varepsilon) \log(1/\varepsilon). \quad (56)$$

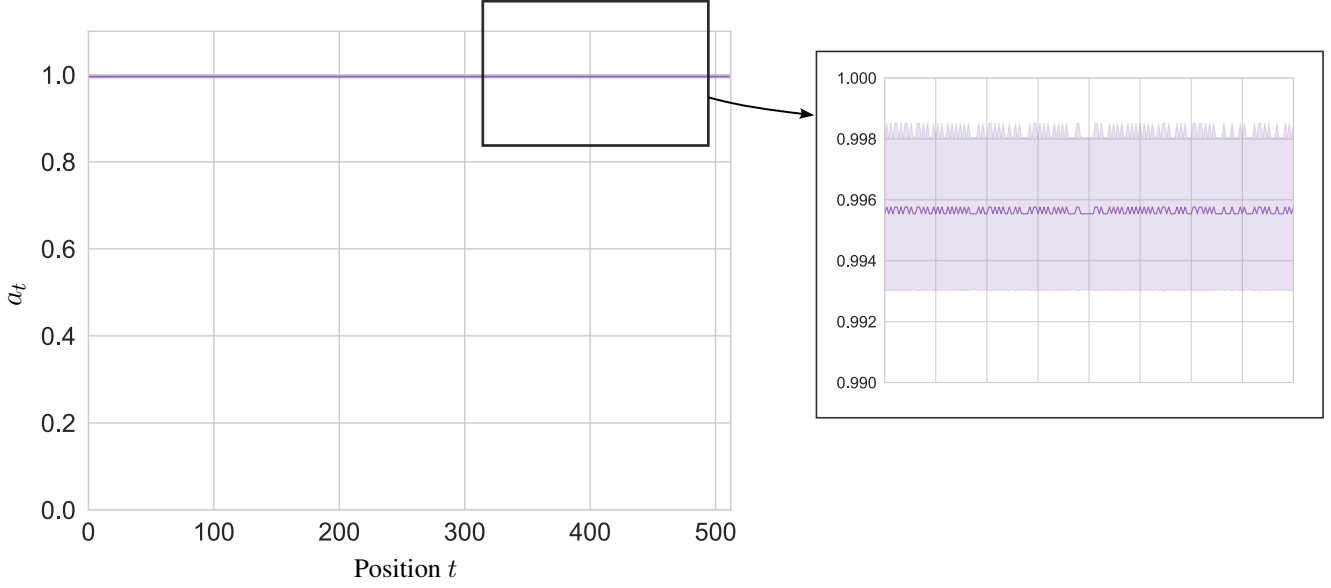
□

Remark. The proof above assumes that the k^{th} -order Markov chain is on a binary state space. However, the result can easily be extended to give the lower bound $d \cdot p \geq \Omega(|\mathcal{X}|^k)$ for larger state spaces, as well as similar scaling results for priors $\text{Dir}(\beta \cdot \mathbf{1})$ for any $\beta > 0$. Furthermore, we believe it should be possible to replace the L_∞ error guarantee in Equation (8) by the KL-divergence between the two distributions without significantly changing the conclusion ($d \cdot p = 2^{\Omega(k)}$).

D. Additional results

D.1. Empirical evidence

Here we provide empirical evidence to the claims in Sec. 4.


 Figure 4. Value of a_t across positions at convergence.

D.2. Switching Markov

Here we detail the switching Markov process more formally:

1. Initialize $t = 0$.
2. Draw a binary Markov kernel P with each row sampled i.i.d. from $\text{Dir}(\beta \cdot \mathbf{1})$.
3. Let $x_t = S$ with probability p_{switch} , or sample $x_t \sim P_{x_{t-k+1}, \cdot}$ with probability $1 - p_{\text{switch}}$.
4. If $x_t = S$, set $t = t + 1$ and go to step 2; if $x_t \neq S$, set $t = t + 1$ and go to step 3.

Fig. 5 illustrates the behavior of Mamba on this process when $p_{\text{switch}} = 0.01$.

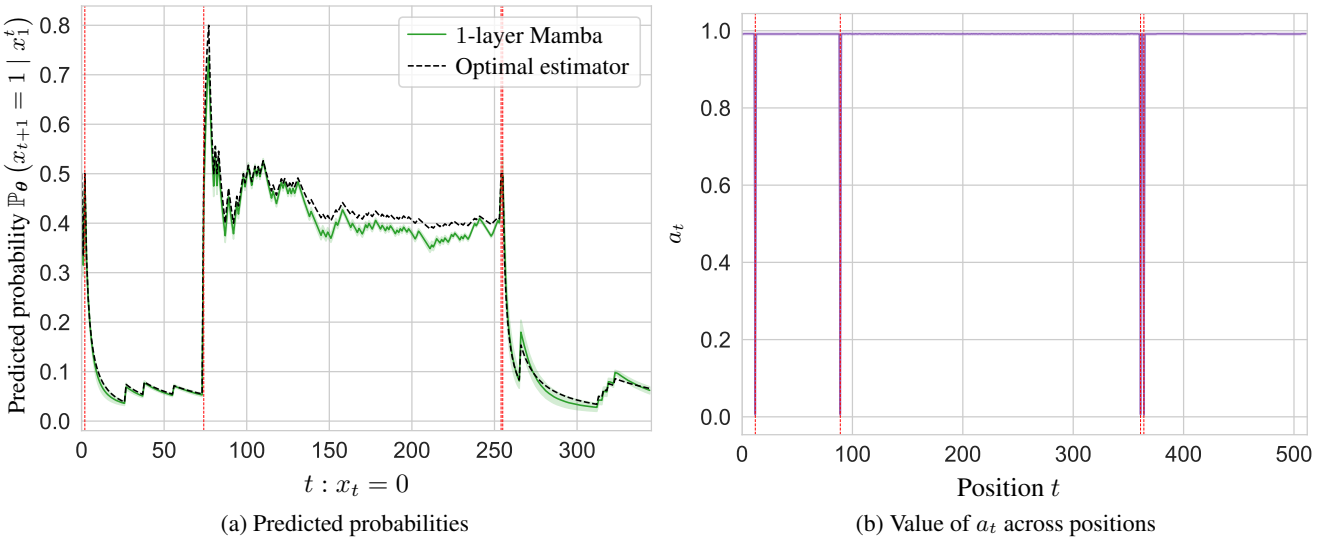


Figure 5. One-layer Mamba on switching Markov data.

E. Model architectures and hyper-parameters

Table 2. Parameters in the Mamba architecture with their shape.

Parameter	Matrix shape
embedding	$2 \times d$
mamba.A	1
mamba.dt	1
mamba.in_proj	$(2ed + 2N + 1) \times d$
mamba.conv1d	$(ed + 2N) \times w$
mamba.out_proj	$d \times (2ed + 2N + 1)$
mlp.fc1	$4d \times d$
mlp.fc2	$d \times 4d$
lm_head	$d \times 2$

Table 3. Settings and parameters for the Mamba model used in the experiments.

Dataset	k -th order binary Markov source
Architecture	Based on the Mamba-2 architecture as implemented in (Dao & Gu, 2024)
Batch size	Grid-searched in $\{16, 32, 64, 128, 256\}$
Accumulation steps	1
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95$)
Learning rate	0.001
Scheduler	Cosine
# Iterations	10000
Weight decay	1×10^{-3}
Dropout	0
Sequence length	Grid-searched in $\{128, 256, 512\}$
Embedding dimension	Grid-searched in $\{2, 4, 8, 16, 32\}$
Mamba layers	1
Heads	1
Convolution window	Between 2 and 6
Repetitions	5

Table 4. Parameters in the transformer architecture with their shape.

Parameter	Matrix shape
transformer.wte	$2 \times d$
transformer.wpe	$N \times d$
transformer.h.ln_1 ($\times \ell$)	$d \times 1$
transformer.h.attn.c_attn ($\times \ell$)	$3d \times d$
transformer.h.attn.c_proj ($\times \ell$)	$d \times d$
transformer.h.ln_2 ($\times \ell$)	$d \times 1$
transformer.h.mlp.c_fc ($\times \ell$)	$4d \times d$
transformer.h.mlp.c_proj ($\times \ell$)	$d \times 4d$
transformer.ln_f	$d \times 1$

Table 5. Settings and parameters for the transformer model used in the experiments.

Dataset	k -th order binary Markov source
Architecture	Based on the GPT-2 architecture as implemented in (Pagliardini, 2023)
Batch size	Grid-searched in $\{16, 32, 64, 128, 256\}$
Accumulation steps	1
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95$)
Learning rate	0.001
Scheduler	Cosine
# Iterations	10000
Weight decay	1×10^{-3}
Dropout	0
Sequence length	Grid-searched in $\{128, 256, 512, 1024\}$
Embedding dimension	Grid-searched in $\{4, 8, 16, 32\}$
Transformer layers	Between 1 and 2 depending on the experiment
Attention heads	1
Repetitions	5